Ruta, A language for the campus Robot

Josue Daniel Castaño Montoya

1. Gramatica del lenguaje Ruta

El lenguaje Ruta permite escribir instrucciones para un Robot que se mueve por un campus universitario. El lenguaje incluye instrucciones para avanzar, girar, detenerse, repetir acciones, evaluar Condiciones y regresar a la base.

1.1 Elementos de la gramatica

- Simbolo inicial <Program>

No terminales: <program>, <statementlist>, <statement>, <assignment>, <forward>, <turn>, <repeat>, <If>, <Condition>,, <RelOp>, <expression>, <Term>, <factor>, <Identifier>, <Number>.

Terminales: robot, forward, turn, left, right, stop, repeat, if, then, else, end, return, base, =, +, -, *, /, <, >, <=, >=, ==, !=, {, }, (,), Indentificadores, números y Saltos de linea.

Gramatica BNF

<Program> ::= Robot <Identifier> NEWLINE <StatementList>

<StatementList> ::= <statement>
| <statement> NEWLINE <statementList>

<Statement> ::= <Assignment>
| <forward>
| <Turn>
| Stop
| <Repeat>
| <If>
| Return Base

<Assignement> ::= <Identifier> = <Expression>

<forward> ::= forward <Expression>

<Turn> ::= turn left
| turn right

<Repeat> ::= repeat <Expression> {
NEWLINE <StatementList> NEWLINE }

<If> ::= if <Condition> then NEWLINE
<StatementList> NEWLINE end
| if <Condition> then NEWLINE
<StatementList> NEWLINE else NEWLINE
<StatementList> NEWLINE end

<Condition> ::= <Expression> <RelOp> <Expression>

Powered by CamScanner

<RelOp> ::= <
| >
| <=
| >=
| ==
| !=

<Expression> ::= <Expression> + <Term>
| <Expression> - <Term>
| <Term>

<Term> ::= <Term> * <factor>
| <Term> / <factor>
| <factor>

<factor> ::= <Number>
| <identifier>
| (<expression>)

<Identifier> ::= letter
| LETTER <identifierTail>

<IdentifierTail> ::= LETTER
| DIGIT
| _
| LETTER <identifierTail>
| DIGIT <identifierTail>
| _ <identifierTail>

<Number> ::= DIGIT
| DIGIT <NumberTail>

<NumberTail> ::= DIGIT
| . DIGIT
| DIGIT <NumberTail>
| . DIGIT <DecimalTail>

<DecimalTail> ::= DIGIT
| DIGIT <DecimalTail>

Powered by CamScanner

Gramatica EBNF

```
Program = "robot", Identifier, NEWLINE, statementList;

statementList = statement, {NEWLINE, statement};

statement = Assignement | forward | Turn | "stop"
          | Repeat | if | "return", "base";

Assignment = Identifier, "=", expression;

forward = "forward", "expression";

Turn = "turn", ("left" | "right");

Repeat = "Repeat", expression, "{", NEWLINE,
         statementList, NEWLINE, "}";

If = "if", condition, "then", NEWLINE, statementList,
     [NEWLINE, "else", NEWLINE, statementList],
     NEWLINE, "end";

Condition = Expression, RelOp, Expression;
RelOp = "<" | ">" | "<=" | ">=" | "==" | "!=";
Expression = Term, {("+" | "-"), Term};
Term = factor, {("*" | "/"), factor};
factor = Number | identifier | "(", expression, ")";
identifier = Letter, {Letter | Digit | "_"};
Number = Digit, {Digit},
         [".", Digit, {Digit}];
Letter = "A"..."Z" | "a"..."z" | "_";
Digit = "0"..."9";
```

Powered by CamScanner

2. Derivacion de un programa válido

robot Tito

stop

2.1 Derivacion por la izquierda

<program>

→ Robot <identifier> NEWLINE <statementlist>

→ Robot Tito NEWLINE <statementlist>

→ Robot Tito NEWLINE <statement>

→ Robot Tito NEWLINE stop

→

2.2 derivacion por la derecha.

<Program>

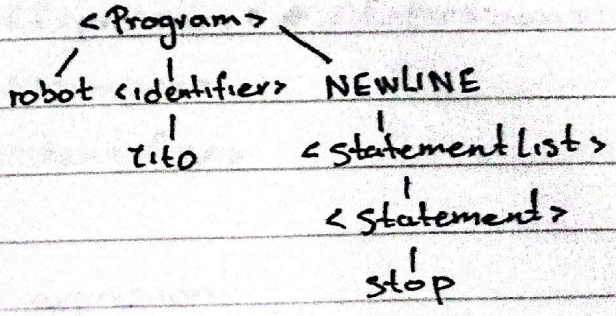
→ robot <identifier> NEWLINE <statementlist>

→ robot <identifier> NEWLINE <statement>

→ robot <identifier> NEWLINE Stop

→ robot Tito NEWLINE stop

Arbol de derivación



Powered by CamScanner

3, Programa invalido

El siguiente programa contiene un error sintáctico

```
robot Tito
speed = 2
forward 10 *
return base.
```

Explicación:

el programa es invalido porque despues del operador * debe aparecer otro factor, como un número, un identificador o una expresion entre parentesis. En este caso, aparece un salto de línea

El analizador sintactico debe señalar la ubicación del error y explicar que falta un factor despues del operador *.

4. Ambigüedad de expresiones aritmeticas

una gramatica ambigua para las expresiones puede escribirse asi:

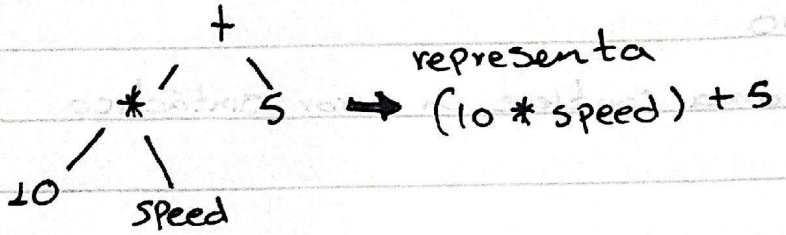
```
<Expression> ::= <Expression> + <Expression>
               | <Expression> * <Expression>
               | <Number>
               | <identifier>
```

Consideremos la expresión

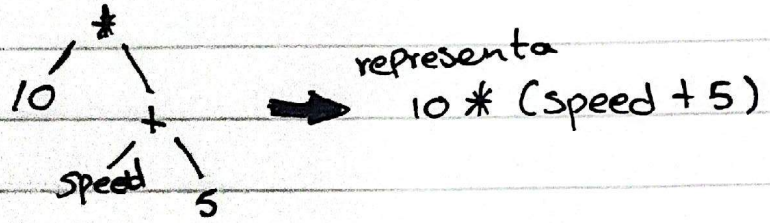
10 * speed + 5

Esta expresion puede interpretarse de dos maneras:

Powered by CamScanner



Arbol 2:



Solucion de la ambigüedad

se separan las expresiones en niveles de precedencia:

<Expression> ::= <Expression> + <Term>
| <Expression> - <Term>
| <Term>

<Term> ::= <Term> * <factor>
| <Term> / <factor>
| <factor>

<factor> ::= <Number>
| <identifier>
| (<Expression>)

Powered by CamScanner

5. Ambigüedad del Else

<statement> ::= if <condition> then <statement>
| if <condition> then <statement>
else <statement>
| <otherstatement>

Consideramos:

```
if A < 10 then
    if B < 5 then
        stop
    else
        forward 1
```

Solución de la ambigüedad

```
if A < 10 then
    if B < 5 then
        stop
    else
        forward 1
    end
end.
```

Powered by CamScanner

6. Tokens, patrones y lexemas

Token	Patrón o Lexema.
ROBOT	robot
FORWARD	forward
TURN	turn
LEFT	left
RIGHT	right
STOP	stop
REPEAT	repeat
IF	if
THEN	then
ELSE	else
END	end
RETURN	return
BASE	base
IDENTIFIER	[A-Za-z_][A-Za-z0-9_]*
NUMBER	[0-9]+(\.[0-9]+)?
ASSIGN	=
ARITHMETIC_OPERATOR	+, -, *, /
RELATIONAL_OPERATOR	<, >, <=, >=, ==, !=
LBRACE	{
RBRACE	}
LPAREN	(
RPAREN	)
NEWLINE	Salto de línea
COMMENT	// seguido de texto hasta el final de línea.

Powered by CamScanner

7. Características de swift relacionadas con el taller

swift permite definir operadores personalizados y establecer su precedencia mediante grupos de precedencia. Además, la clasificación de ciertos operadores como Prefijos, infijos o sufijos puede depender de los espacios en blanco

En este taller, la precedencia de los operadores de Ruta se implementa manualmente mediante los niveles <Expression>, <Term> y <factor>. De esta forma, el procesador reconoce primero las multiplicaciones y divisiones y despues las sumas y restas.